AI-Powered Cybersecurity Incident Response Assistant

Investigate, classify, prioritise and respond to network security incidents with hybrid RAG, machine learning and a multi-agent LLM workflow

Capstone Project 8 · CIC-IDS2017

Nalin · October 2026

PROBLEM STATEMENT

Analysts drown in network alerts

- Security teams monitor traffic for **brute force, DoS / DDoS, web attacks, botnets, infiltration and port scans**
- Every alert needs the same four answers (right)
- Finding them by hand across **millions of flow records** is slow and inconsistent
- Urgent incidents can **wait behind routine ones**
- **Goal:** cut the time from alert to decision, with answers analysts can trust

What attack is it?
attack type and confidence

How dangerous is it?
risk, anomaly and P1–P4 priority

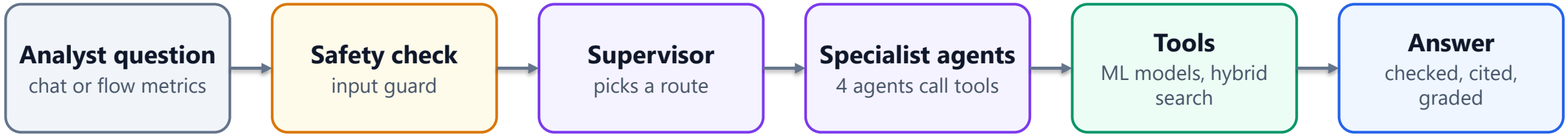
Has it happened before?
similar past incidents

What should we do?
investigation and response steps

Data: CIC-IDS2017, 2.8 million labelled network flows, 15 traffic types

SOLUTION OVERVIEW

One question in, an evidence-based answer out



Hybrid RAG search

97,549 past incidents: keyword + semantic search, fused, with exact filters

ML classification & risk

Attack type (15 classes), intrusion risk, anomaly score

Explainable decisions

SHAP shows which metrics drove each prediction, vs normal traffic

Prioritisation

Risk + severity + anomaly → P1–P4, for one flow, a batch or a CSV upload

Multi-agent investigation

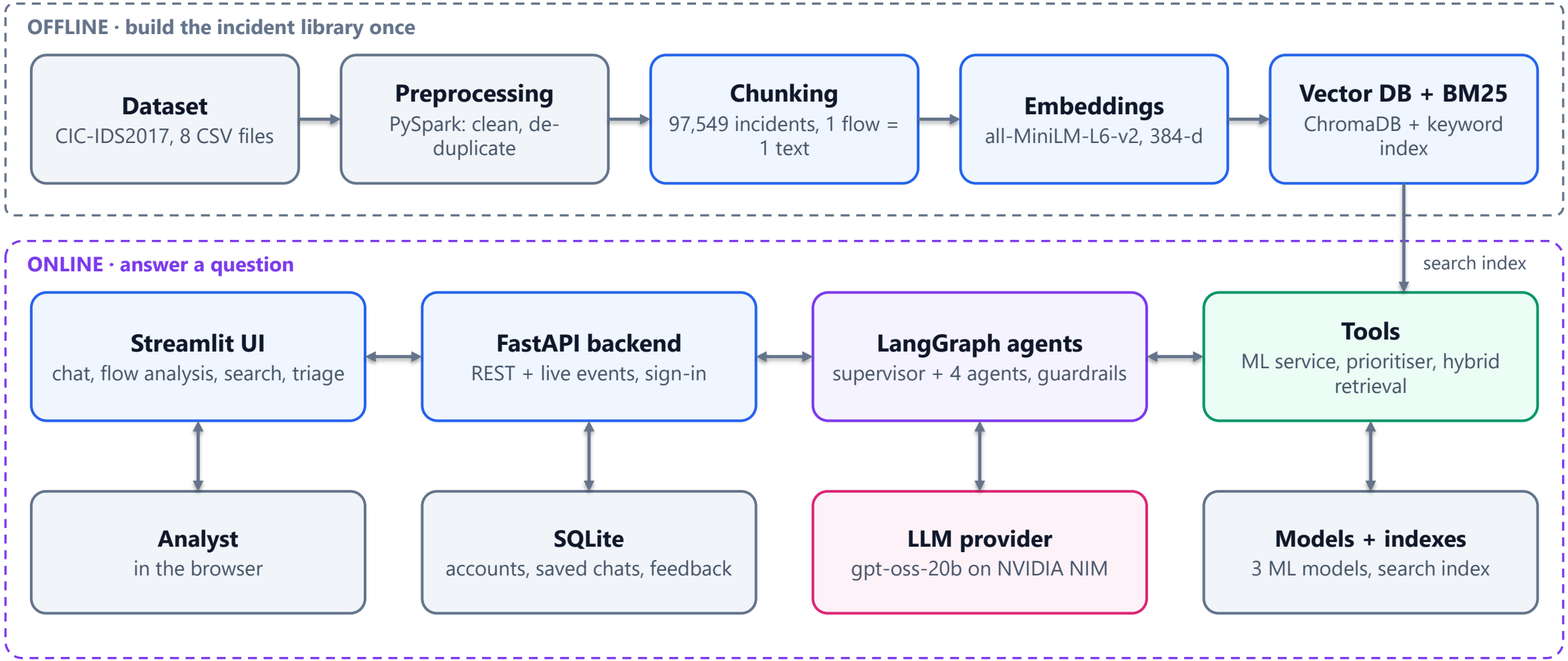
LangGraph supervisor + 4 specialist agents, some running in parallel

Safety & quality

Guardrails, LLM-as-Judge, DeepEval, analyst feedback and safe retraining

HOW THE PARTS FIT TOGETHER

Architecture and data flow



CANADIAN INSTITUTE FOR CYBERSECURITY

Dataset: CIC-IDS2017

2,830,743
network flows in 8 CSV files

15
labels: BENIGN + 14 attack types

5 days
of lab traffic, 3–7 July 2017

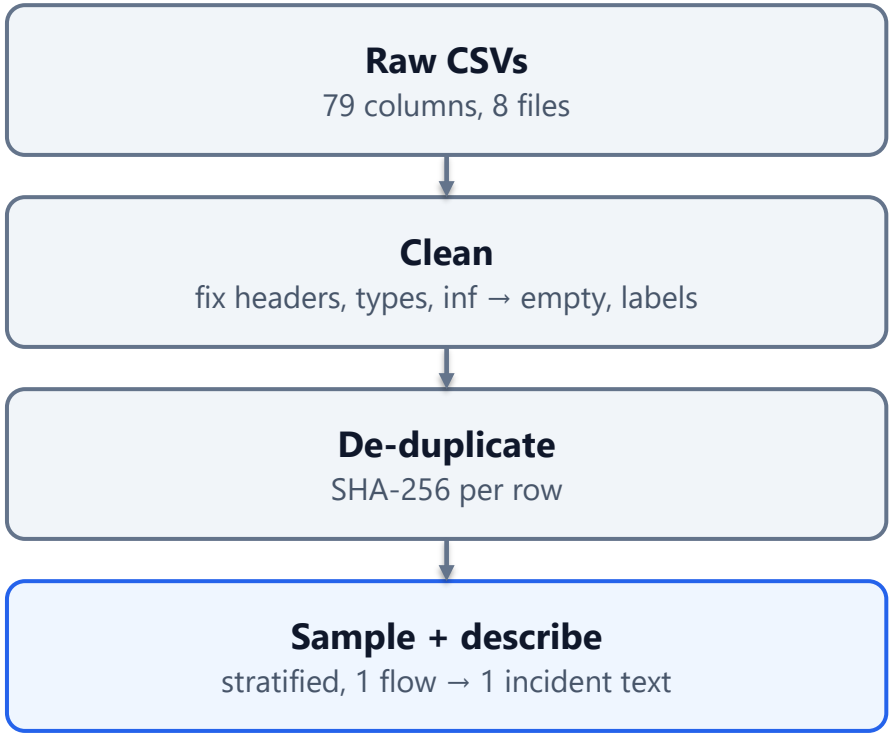
2,236,725
flows after cleaning and de-duplication

97,549
incidents in the search library

24
flow metrics used by the models

Labels include

DoS (Hulk, GoldenEye, Slowloris, Slowhttptest), DDoS, PortScan, SSH- and FTP-Patator, Web attacks (brute force, XSS, SQL injection), Bot, Infiltration, Heartbleed



Limitation

No IP, protocol or timestamp in this release

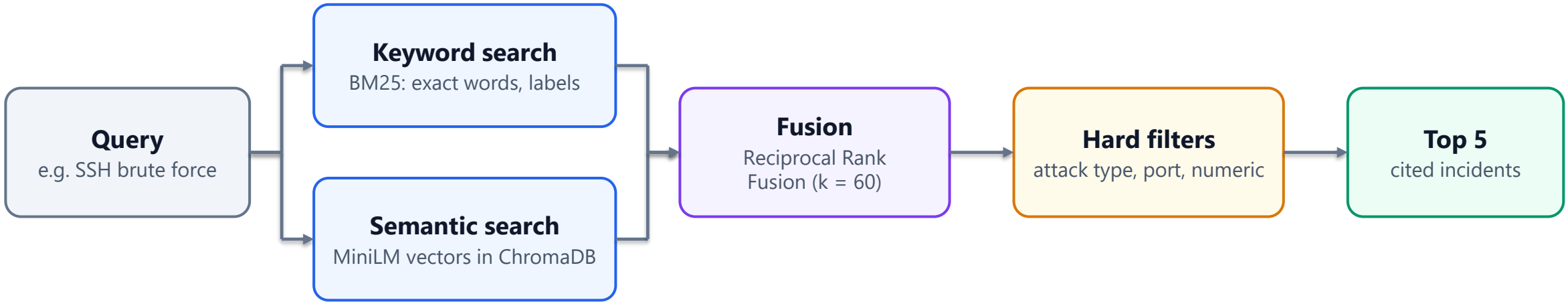
BUILT WITH

Technology stack

Component	Technology
Language	Python 3.10
LLM	openai/gpt-oss-20b on NVIDIA NIM (switchable to Gemini or Claude in .env)
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384 dimensions)
Vector database	ChromaDB (persistent, cosine distance)
Keyword search	rank-bm25 (BM25Okapi), fused with Reciprocal Rank Fusion
Agent framework	LangGraph, LangChain
Machine learning	scikit-learn (Random Forest, gradient boosting, Isolation Forest), SHAP
Data processing	PySpark, Pandas, PyArrow
Backend	FastAPI, Uvicorn (REST + Server-Sent Events, 26 endpoints)
Front end	Streamlit, Plotly
Storage	SQLite: accounts, saved conversations (LangGraph SqliteSaver), feedback
Evaluation	DeepEval, LLM-as-Judge, pytest

FINDING SIMILAR PAST INCIDENTS

Retrieval strategy: hybrid RAG

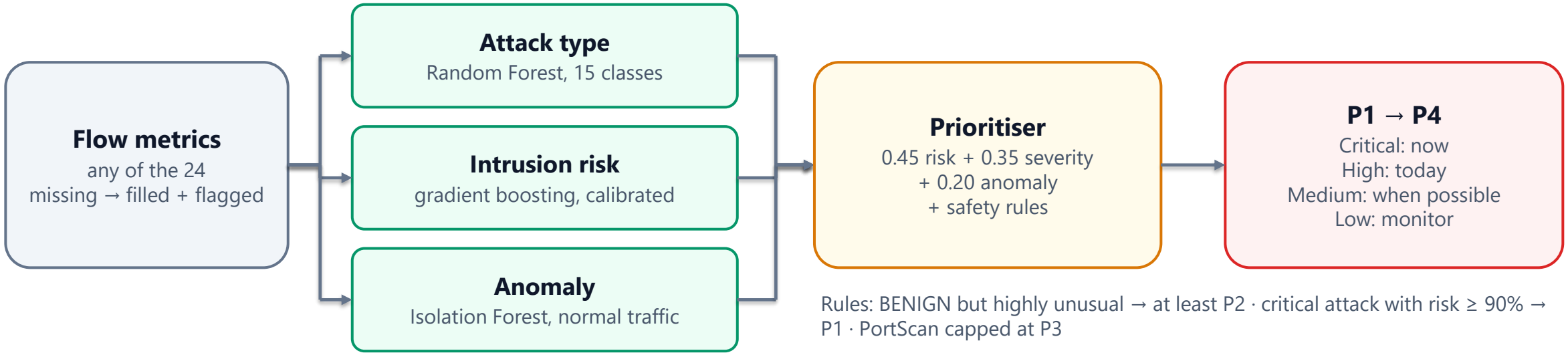


Method (12 labelled queries)	Hit@5	Precision@5	MRR
BM25 keyword	0.583	0.567	0.583
Vector (semantic)	0.833	**0.833**	**0.833**
Hybrid (RRF)	**0.917**	0.733	0.792

- Each incident is one flow written as a short text (one chunk)
- Agents set the filters: 'SSH brute force' → **SSH-Patator**
- Numbers in the question become filters: 'more than 10000 packets per second'

CLASSIFY, SCORE, EXPLAIN, PRIORITISE

Machine-learning models



Rules: BENIGN but highly unusual → at least P2 · critical attack with risk ≥ 90% → P1 · PortScan capped at P3

SHAP explanations

Which metrics drove each prediction, compared with normal traffic

0.986
classifier accuracy (macro F1 0.839)

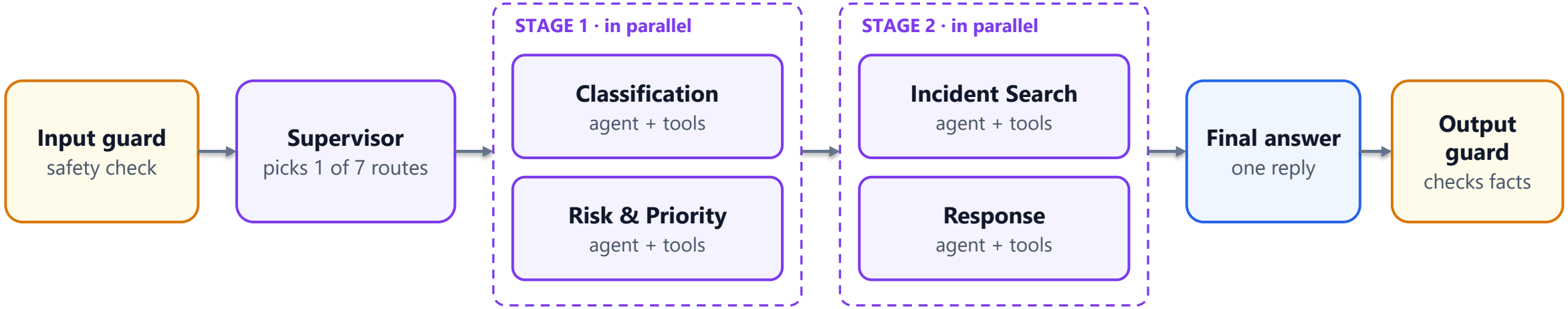
0.9997
risk model ROC-AUC, 0.7% false alarms

0.855
anomaly detector ROC-AUC

71%
unseen Infiltration flagged as anomalous

ONE TURN THROUGH THE SUPERVISOR GRAPH

Multi-agent workflow: LangGraph



classify	risk	search	recommend	investigate	general	out of scope
Classification	Risk & Priority	Incident Search	Response	all 4 agents, 2 stages	answer directly	fixed refusal

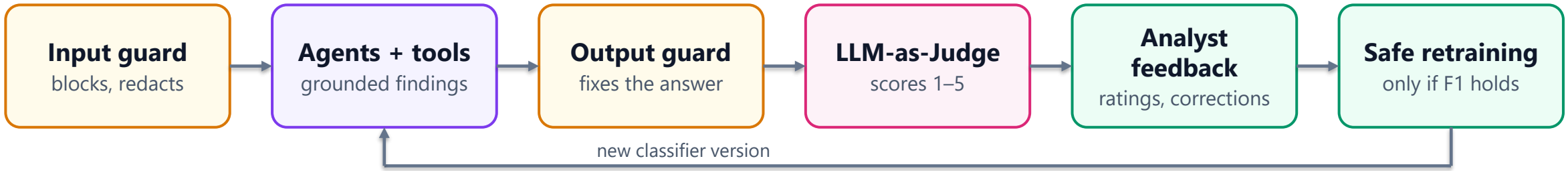
Direct reply
A single agent writes the answer itself: one LLM call saved

Follow-up
Missing duration, packets or bytes → asks the user, never guesses

Bounded and visible
At most 6 tool rounds per agent; every step streamed live to the UI

GUARDRAILS, LLM-AS-JUDGE, FEEDBACK

Safety and quality



Input guard

- Blocks prompt injection and attack requests; defensive questions allowed
- Redacts keys, passwords, emails, card numbers

Output guard

- Marks incident IDs no tool returned
- Flags wrong figures and priorities
- Removes dangerous commands, adds caveats

LLM-as-Judge + DeepEval

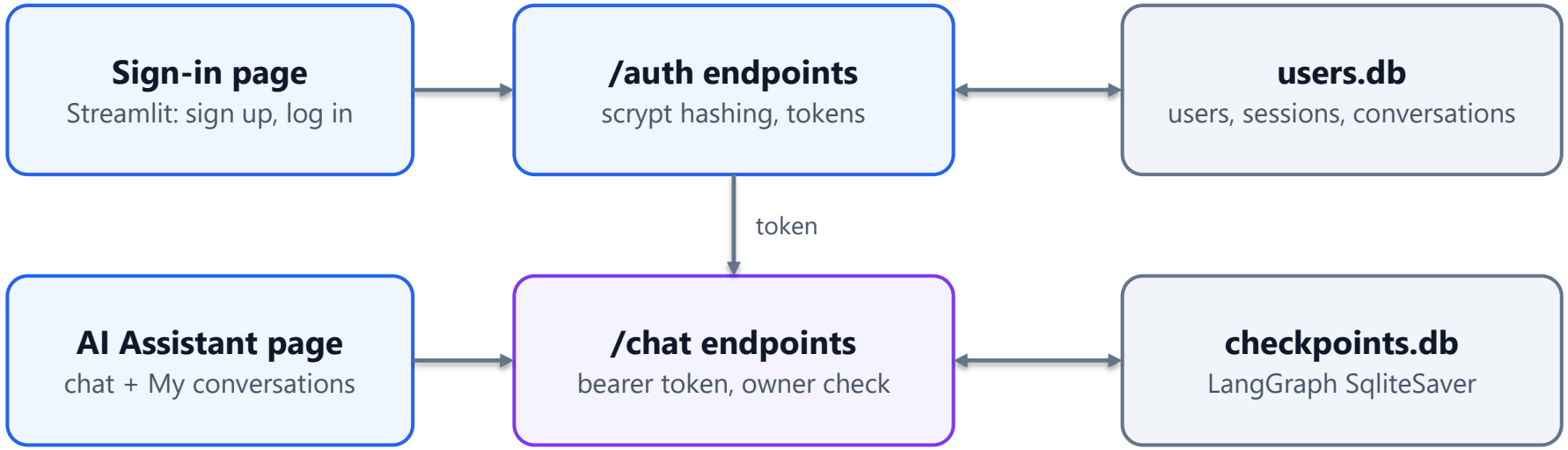
- Faithfulness, correctness, relevance, actionability, safety
- Rule caps; calibration 6/6
- DeepEval: 24 cases, 5 metrics + exact checks

Feedback loop

- Thumbs up / down, comments, attack-type corrections
- Corrections shown in later answers
- Exportable for review

EACH ANALYST KEEPS THEIR OWN HISTORY

User accounts and saved conversations



- Sign up / log in / log out
- Chats saved per user
- Resume after restart

Security

- Passwords hashed with scrypt; tokens random, only a SHA-256 digest stored
- Tokens expire after 7 days; logout revokes them; 5 failed logins in 5 min → blocked

Conversations

- List, resume, rename and delete your own chats
- Another user's chat returns 404; feedback records who gave it

MEASURED, THEN IMPROVED

Evaluation results and performance

0.97 faithfulness (DeepEval)

0.97 answer relevancy

99% exact checks passed

0.93 safety and caveats

0.917 hybrid search Hit@5

413 automated tests, all passing

Quality (24 cases)	Before	After
Exact checks	98%	**99%**
Faithfulness	0.95	**0.97**
Answer relevancy	0.95	**0.97**
Recommendation	0.88	**0.92**
Safety	0.92	**0.93**
Contextual relevancy	0.85	0.75*
Judge vs DeepEval agreement	67%	**81%**

Response time	Before	After
Full investigation	72–133 s	**54–68 s**
Simple question	16–23 s	**7–13 s**

- Investigations run agents in **2 parallel stages**
- Single-agent questions **skip the rewrite**
- Evaluation found and fixed 4 real defects

* The evaluator misread 2 correctly filtered searches (right attack type, right incidents)

WHAT WAS HARD AND WHAT WE LEARNED

Challenges and learnings

Challenges

- Dataset has no IP, protocol or timestamp: filters built, but only attack type, port and numbers have data
- Free LLM tiers: Claude credit ran out, Gemini 20 requests/day → provider-agnostic, gpt-oss-20b on NIM
- Small-model quirks: broken JSON, invented IDs, extra general knowledge
- Partial input: analysts know 3–5 of the 24 metrics
- Latency: 1–3 minutes per investigation → now about 1 minute with parallel agents

Learnings

- Wrap the LLM in deterministic checks: guardrails, rule caps and exact checks catch what it misses
- Measure early: evaluation exposed four real defects
- Route, don't pipeline; run independent agents in parallel
- Filters beat clever ranking for precise retrieval
- Show the work: a live agent trace builds trust
- Keep the LLM a configuration choice, not a dependency

NEXT STEPS

Future work

- Use the **GeneratedLabelledFlows** release: IP, protocol and timestamp filters; multi-flow incidents
- **Stronger evaluator model** and a larger labelled case set
- **Docker / docker-compose** for one-command deployment
- **CI pipeline** running the 413 tests on every push
- Stay signed in across a browser refresh; **HTTPS** for non-local use
- Cache repeated tool lookups to cut LLM calls further

Repository

github.com/nlnkrthk/Cybersecurity-assistant

README · Architecture diagram
Design document · API documentation
Dataset details · Evaluation report
Sample data · 413 tests

Thank you

Questions?

github.com/nlnkrthk/Cybersecurity-assistant